

Unidad 3 — Análisis descriptivo potenciado por IA

Curso: Toma de decisiones basadas en datos con IA · Explicación completa del material

Esta unidad baja por primera vez al motor técnico de la IA empresarial: el **aprendizaje automático supervisado** (*supervised machine learning*). Cubre cinco bloques: (1) el flujo de trabajo de 7 pasos que va desde recopilar datos hasta mantener el modelo en producción, (2) las tres familias de algoritmos para datos estructurados —regresión lineal, árboles de decisión y métodos de conjunto—, (3) los modelos para datos no estructurados —aprendizaje profundo, PLN, visión por computador— más el aprendizaje por refuerzo como paradigma alternativo, (4) las métricas de evaluación para regresión y clasificación, y (5) los cinco errores estratégicos más comunes. Cierra con dos casos aplicados: el modelado del valor de vida del cliente (LTV) y la predicción de fuga de clientes en NovaMarket.

¿Dónde encaja en el curso? Si los módulos anteriores te dieron el **mapa** del proyecto (CRISP-DM) y la **materia prima** (las cinco V del Big Data, la veracidad, el ground truth), esta unidad te muestra el **motor** que corre dentro de las fases de Modelado y Evaluación de CRISP-DM. Todo lo que veas aquí ocurre "dentro" de las fases 3 a 6 de esa metodología.

Una advertencia de entrada: aunque el título dice "análisis **descriptivo**", el contenido real de la unidad es análisis **predictivo**. Lo desarrollo en las Notas de discrepancia al final, pero tenlo presente desde ya para que el título no te confunda en el examen.

Conceptos clave: el vocabulario de arranque

El PDF abre con siete definiciones que funcionan como diccionario mínimo de la unidad. Las presento aquí con su intuición; cada una se desarrolla a fondo más adelante.

Características (*features*) son las variables de entrada que el modelo usa como señales para predecir: el tamaño de una casa, la edad del cliente. En Fibromin, serían las lecturas de tus sensores: temperatura del rodamiento, vibración RMS, horas de operación desde la última mantención. Son las "pistas" que le das al modelo.

Etiquetas (*labels*) son el resultado que el modelo intenta predecir a partir de las características: la variable objetivo. En Fibromin: "¿este equipo falló dentro de los 30 días siguientes? sí/no". La etiqueta es la respuesta correcta del problema, y conocerla en los datos históricos es lo que define al aprendizaje *supervisado*.

Regresión (*regression*) es el tipo de problema donde la etiqueta es un **número continuo**: ingresos en dólares, precio de una vivienda, horas de vida útil restante de un motor. La pregunta es "¿cuánto?".

Clasificación (*classification*) es el tipo de problema donde la etiqueta es una **categoría discreta**: "fraude" o "no fraude", "falla" o "no falla", "cliente en riesgo" o "cliente estable". La pregunta es "¿cuál?". Esta distinción —regresión vs. clasificación— es la primera decisión técnica de cualquier proyecto, porque determina qué algoritmos puedes usar y con qué métricas evaluarás.

Conjunto de prueba (*test set*) es la porción de los datos históricos que **no** se usa para entrenar y se reserva exclusivamente para evaluar el rendimiento final del modelo. Es el examen sorpresa: preguntas que el alumno nunca vio.

Inferencia (*inference*) es la fase en que el modelo, ya entrenado y "congelado", recibe datos nuevos y genera una predicción o puntuación. Aquí el modelo ya no aprende: solo aplica lo aprendido.

Sobreajuste (*overfitting*) es el error que ocurre cuando un modelo funciona muy bien con los datos de entrenamiento pero falla al generalizar con datos nuevos. Es el alumno que memorizó el facsímil y reprueba el examen real. Es el enemigo número uno de todo lo que viene.

El proceso de aprendizaje automático supervisado

La idea central: aprender con un solucionario

El aprendizaje supervisado es el paradigma más utilizado en proyectos empresariales de IA porque su principio es simple: **aprender a partir de ejemplos históricos donde ya se conoce el resultado correcto**. El sistema recibe datos de entrada junto con su "verdad básica" o etiqueta (**ground truth**), y su objetivo es descubrir las **relaciones estadísticas** que permitan predecir correctamente casos futuros.

La analogía que vamos a expresar durante toda la unidad: estudiar con un **facsímil resuelto**. Te entregan miles de preguntas con su respuesta correcta (los datos etiquetados). Estudias hasta detectar los patrones que conectan pregunta y respuesta. El día del examen (el conjunto de prueba) te llegan preguntas que **nunca viste**: si aprendiste los patrones, las respondes bien (el modelo *generaliza*); si solo memorizaste las preguntas del facsímil, repruebas (el modelo *sobreajustó*).

Conexión con el módulo anterior: ya habíamos separado **Veracidad** (¿puedo confiar en mis datos en general?) de **Ground Truth** (la respuesta correcta verificada). Aquí el ground truth toma protagonismo: es la columna de etiquetas con la que el modelo estudia y, después, la vara contra la que se le toma examen. Sin ground truth confiable no hay aprendizaje supervisado posible —y un ground truth con baja veracidad (etiquetas mal puestas) significa entrenar con un solucionario lleno de errores.

¿Por qué domina este paradigma en las empresas? Porque gran parte de los problemas analíticos ya viene con etiquetas históricas gratis: si quieres predecir la **fuga de clientes** (*churn*), tienes registros de quienes efectivamente se fueron; si quieres detectar **fraude**

financiero, tienes transacciones ya clasificadas como fraudulentas o legítimas. En Fibromin, cada falla histórica de un equipo es una etiqueta lista para usar.

El punto metodológico que el PDF subraya —y que es examinable—: aunque desde fuera el proceso parece automático, el éxito **no depende únicamente del algoritmo**, sino de la calidad metodológica de cada fase del flujo. Ese flujo tiene **siete pasos**:

Paso 1 — Recopilar y organizar datos

La primera etapa consiste en recopilar y estructurar datos históricos relevantes para el problema de negocio, y suele **consumir la mayor parte del tiempo total del proyecto**, porque los datos empresariales rara vez están listos para usarse. Esto calza con lo que vimos en CRISP-DM: las fases de comprensión y preparación de datos son las devoradoras de horas.

Los algoritmos exigen datos consistentes, ordenados y comparables. El estándar para lograrlo es el concepto de **Tidy Data** ("datos ordenados"), que impone tres reglas:

1. Cada **fila** representa una observación.
2. Cada **columna** corresponde a una variable.
3. Cada **celda** contiene un único valor.

Nota breve (prerrequisito de origen): el término fue formalizado por Hadley Wickham (2014) en el paper "Tidy Data", y hoy es el estándar de facto en ciencia de datos.

En Fibromin, una tabla tidy se vería así: cada fila es **una lectura de un equipo en una fecha** ("bomba B-103, 2025-08-14"); las columnas son las variables (temperatura, vibración, presión, horas de operación, ¿falló en los 30 días siguientes?); y ninguna celda mezcla dos cosas (nada de "85°C / alta vibración" metido en una sola celda).

El principio parece trivial, pero tiene consecuencias profundas: si las estructuras son ambiguas o inconsistentes, el modelo **aprende relaciones erróneas** o simplemente no detecta patrones útiles. Los problemas frecuentes que aparecen en esta etapa son: datos incompletos, registros duplicados, errores de digitación, variables inconsistentes, formatos incompatibles y ausencia de trazabilidad histórica.

Dos ideas más cierran el paso. Primero, en entornos reales gran parte del trabajo del científico de datos es justamente **transformar información operativa en datos aptos para ML**. Segundo —y esto es una frase de examen—: la calidad de un modelo depende mucho más de la **relevancia** de los datos que de la **cantidad absoluta** almacenada. Conexión directa con el módulo de Big Data: el Volumen sin Veracidad ni Valor no sirve de nada.

Paso 2 — Definir características y etiquetas

Con los datos organizados, hay que separar explícitamente qué es **señal de entrada** (características / *features*) y qué es **objetivo a predecir** (etiqueta / *label*). Las etiquetas pueden ser categorías, clases, probabilidades o valores numéricos continuos, según el problema. Las características son las variables que el modelo usará como pistas predictivas.

Los ejemplos del curso: en un sistema de **riesgo crediticio**, las features serían edad, ingresos, historial de pagos y comportamiento financiero; en un modelo de **fuga de clientes**, frecuencia de compra, número de reclamos e interacción digital. En Fibromin: vibración, temperatura, corriente del motor y antigüedad del equipo como features; "falla en 30 días (sí/no)" como label si planteas clasificación, o "horas de vida útil restante" como label si planteas regresión. Fíjate que **el mismo problema de negocio puede formularse de las dos maneras**, y esa formulación define todo lo que viene después: algoritmos y métricas.

Aquí aparece una idea fundamental del PDF: **los modelos no "entienden" el negocio; solo aprenden relaciones estadísticas entre variables**. El modelo no sabe qué es una bomba ni qué es un cliente: ve columnas de números y busca correlaciones. Por eso la selección de características es crítica: variables irrelevantes agregan **ruido**; variables útiles mejoran significativamente la capacidad predictiva. Y otra frase de examen: en muchos proyectos, **la calidad del diseño de variables tiene más impacto que la sofisticación del algoritmo**. A esta disciplina de construir buenas variables se le llama en la práctica **ingeniería de características** (*feature engineering*): un Random Forest con features mediocres pierde contra una regresión lineal con features bien diseñadas.

Paso 3 — Dividir en conjuntos de entrenamiento y prueba

Un modelo útil debe ser capaz de **generalizar**: funcionar correctamente sobre datos que nunca vio. Para comprobarlo, el conjunto total de datos se divide en:

- **Conjunto de entrenamiento** (*training set*): con el que el algoritmo aprende los patrones.
- **Conjunto de prueba** (*test set*): reservado exclusivamente para evaluar el desempeño final.
- En algunos proyectos, un **conjunto de validación** (*validation set*) adicional: se usa durante el desarrollo para comparar configuraciones y ajustar hiperparámetros **sin contaminar** el test, que debe quedar virgen hasta la evaluación final.

¿Por qué separar? Porque evita el problema más peligroso del aprendizaje automático: que el sistema **memorice** los ejemplos históricos sin aprender reglas generalizables. Es exactamente la diferencia entre el alumno que entendió la materia y el que se aprendió el facsímil de memoria: ambos sacan 100% en las preguntas conocidas, pero solo el primero aprueba con preguntas nuevas. La nota de entrenamiento siempre es engañosamente optimista; la nota que importa es la del test.

Las divisiones más comunes son **70/30 u 80/20** (entrenamiento/prueba), aunque varían según el tamaño del conjunto de datos y el tipo de problema.

El matiz crítico: en **escenarios temporales** (predicción financiera, demanda, mantenimiento predictivo), la división debe **respetar la secuencia cronológica**. Si mezclas datos al azar, el modelo podría "ver el futuro" durante el entrenamiento, generando resultados artificialmente optimistas. Este fenómeno se llama **fuga de datos** (*data leakage*). En Fibromin: entrena con 2022–2024 y prueba con 2025; jamás al revés ni con una mezcla aleatoria, porque si el modelo entrenó con lecturas de marzo de 2025 y lo evalúas prediciendo febrero de 2025, le estás dejando copiar las respuestas del futuro. En

producción ese futuro no existirá, y el desempeño real se derrumbará respecto del que mediste.

Paso 4 — Utilizar algoritmos para encontrar el mejor modelo

Aquí el PDF exige distinguir tres términos que la gente usa como sinónimos y **no lo son**:

- El **algoritmo** es el procedimiento matemático utilizado para aprender.
- El **entrenamiento** (*training*) es el proceso mediante el cual ese algoritmo ajusta parámetros usando los datos históricos.
- El **modelo** es el resultado final aprendido.

Analogía: el algoritmo es la **receta**, el entrenamiento es el acto de **cocinar** con tus ingredientes (tus datos), y el modelo es el **plato terminado**. La misma receta con ingredientes distintos produce platos distintos: el mismo algoritmo entrenado con los datos de Fibromin y con los de un banco produce dos modelos que no se parecen en nada.

Durante el entrenamiento, el sistema intenta **minimizar el error** entre las predicciones generadas y las etiquetas reales (técnicamente, minimiza una *función de pérdida* que cuantifica ese error). Para lograrlo se realizan múltiples iteraciones experimentales donde se ajustan **hiperparámetros**, se seleccionan variables y se prueban técnicas de validación. Vale distinguir: los **parámetros** son los valores internos que el modelo *aprende* solo (por ejemplo, los coeficientes de una regresión); los **hiperparámetros** son las perillas que *tú fijas antes* de entrenar (la profundidad máxima de un árbol, el número de árboles de un bosque). Ajustar hiperparámetros es como decidir cuántas horas estudiar y con qué método, antes de ponerte a estudiar.

Y la idea estrella del paso: **no existe un algoritmo universalmente superior**. La elección depende del volumen de datos, la complejidad del fenómeno, el nivel de interpretabilidad requerido y las restricciones operacionales del negocio. *Nota breve*: esta intuición tiene respaldo formal en el teorema "No Free Lunch" de la teoría de aprendizaje. Por eso, en prototipos empresariales lo habitual es **comparar múltiples modelos** antes de seleccionar uno definitivo —exactamente lo que hará NovaMarket en el caso final.

Paso 5 — Evaluar el modelo final

Antes de llevar un modelo a producción hay que responder con rigor una pregunta:

¿funciona bien sobre datos nuevos? Para eso se usan métricas específicas según el tipo de problema: en **regresión**, RMSE y R-cuadrado; en **clasificación**, exactitud, precisión, recall, F1 y ROC-AUC. Todas se desarrollan en detalle en la sección de Evaluación; aquí lo importante es el criterio estratégico.

Ese criterio dice: evaluar **no es solo obtener un buen número**. También hay que revisar estabilidad, sesgos, sensibilidad y comportamiento por segmentos específicos. Y sobre todo: en contextos empresariales, **distintos errores tienen costos diferentes**. En detección de fraude, un **falso negativo** (fraude que pasó colado) puede ser carísimo; en campañas de marketing, un **falso positivo** (contactar a alguien que no correspondía) suele ser aceptable. En Fibromin la asimetría es brutal: un falso negativo es una falla no detectada que puede significar una parada de planta de millones; un falso positivo es una

inspección innecesaria que cuesta unas horas-hombre. Por eso **la métrica correcta depende siempre del contexto operacional y del impacto económico de cada tipo de error** — frase literalmente examinable.

Paso 6 — Implementación (inferencia)

Cuando el modelo alcanza un desempeño aceptable, se **despliega** para que lo usen personas o sistemas. Esta etapa es la **inferencia**: el modelo ya no aprende, solo recibe datos nuevos y genera predicciones con lo que aprendió. El conocimiento queda congelado en el estado del último entrenamiento.

La inferencia puede ejecutarse de dos formas, según la necesidad del negocio:

- **En tiempo real mediante APIs** (*Application Programming Interfaces*: interfaces que permiten que otros sistemas consulten el modelo al instante). Ejemplo del PDF: un sistema antifraude debe responder **en segundos**, antes de aprobar la transacción.
- **Por lotes** (*batch processing*): el modelo se ejecuta periódicamente sobre un grupo de casos. Ejemplo: un modelo de segmentación comercial que corre una vez al día. En Fibromin podrías tener ambos: scoring batch nocturno de toda la flota de equipos, y consulta en tiempo real cuando un sensor dispara una alerta.

La advertencia del curso: la implementación es **mucho más compleja de lo que aparenta**, porque exige integrar el modelo con bases de datos, sistemas transaccionales, plataformas web e interfaces de usuario. Conclusión examinable: **un modelo técnicamente excelente puede fracasar si no logra incorporarse al flujo operativo de la organización**. Es la distinción output/outcome que ya manejamos: el modelo y sus predicciones son el *output*; el *outcome* —la decisión mejorada, la parada evitada, el cliente retenido— solo ocurre si ese output llega integrado a quien decide.

Paso 7 — Mantención y reentrenamiento

Uno de los errores más comunes es creer que el ciclo termina con la implementación. En realidad, **los modelos envejecen**: los patrones de comportamiento cambian por transformaciones económicas, nuevas conductas de los clientes, movimientos de la competencia, cambios regulatorios o modificaciones tecnológicas. El PDF llama a este fenómeno **drift o deriva de datos**.

Aquí conviene rescatar el matiz que ya vimos en el módulo anterior, porque el curso lo agrupa todo bajo un solo nombre:

- **Data drift**: cambia la distribución de las **entradas** (tus sensores ahora leen en otro rango porque la máquina se desgastó).
- **Concept drift**: cambia la **regla** misma, la relación entrada→salida (el patrón de vibración que antes anticipaba la falla ya no la anticipa, porque cambió la física del proceso o el modo de operación).

Ambos degradan el modelo, pero se diagnostican y corrigen distinto. Para efectos del examen de esta unidad, "drift o deriva de datos" a secas es la respuesta esperada; el matiz te sirve para entender y para preguntas más finas.

La consecuencia operativa: los modelos deben **monitorearse continuamente** para detectar caídas de desempeño, y cuando la precisión baja, **reentrenarse con datos más recientes**. En organizaciones maduras esto se institucionaliza como **MLOps** (*Machine Learning Operations*): un ciclo permanente donde entrenamiento, monitoreo, despliegue y actualización son un proceso continuo de mejora analítica —el equivalente al DevOps del software, aplicado a modelos. La ironía que te va a gustar: el modelo de mantenimiento predictivo de Fibromin **también necesita su propio mantenimiento predictivo**. Monitorear el modelo es vigilar al vigilante.

Síntesis del flujo: el valor de un proyecto supervisado no está en un paso, sino en la cadena completa: datos ordenados → variables bien definidas → división honesta → entrenamiento comparativo → evaluación con la métrica correcta → integración operativa → mantención permanente. Romper cualquier eslabón invalida el resto.

Algoritmos populares de aprendizaje automático

Existen cientos de algoritmos, pero una parte importante de los proyectos reales se resuelve con unas pocas familias bien conocidas. El objetivo del curso no es que domines la matemática de cada uno, sino que identifiques **qué problemas resuelve cada familia, qué ventajas tiene y qué limitaciones presenta** en contextos organizacionales.

Los algoritmos se diferencian según cinco criterios (el PDF los lista de la A a la E):

1. **El tipo de problema que resuelven** (regresión, clasificación, otros).
2. **La forma en que representan patrones** (una ecuación, reglas jerárquicas, redes de neuronas).
3. **El nivel de interpretabilidad** (¿puedo explicar por qué predijo lo que predijo?).
4. **El costo computacional** (tiempo y hardware para entrenar y para inferir).
5. **La precisión que alcanzan**.

Y la tesis transversal, examinable: **en entornos empresariales no siempre gana el modelo más complejo**. Muchas veces se prefiere un algoritmo ligeramente menos preciso pero más fácil de explicar, mantener o implementar. La precisión es una dimensión más, no la única.

Regresión lineal (1 de 3)

La figura del PDF muestra un diagrama de dispersión con puntos de datos y una recta azul atravesándolos: esa recta es el modelo. La **regresión lineal** (*linear regression*) es el punto de partida conceptual de muchos modelos avanzados: busca la **relación matemática lineal** entre las variables de entrada y una salida numérica, ajustando la línea (o el hiperplano, con varias variables) que mejor representa los datos observados.

La fórmula (el PDF no la escribe, pero es indispensable dominarla):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Desglose símbolo por símbolo:

- $\hat{y}$ ("y sombrero"): el valor **predicho** por el modelo (el sombrero distingue la predicción del valor real y).
- $x_1, x_2, \dots, x_n$: las **características** (features) de entrada.
- $\beta_1, \dots, \beta_n$: los **coeficientes** (*coefficients*), los parámetros que el modelo aprende. Cada β_i dice cuánto cambia la predicción cuando x_i aumenta en una unidad, manteniendo el resto constante.
- β_0 : el **intercepto**, el valor base de la predicción cuando todas las features valen cero.

¿Cómo encuentra el algoritmo los β ? Con el método de **mínimos cuadrados ordinarios** (*Ordinary Least Squares, OLS*): elige los coeficientes que **minimizan la suma de los errores al cuadrado**, $\sum_{i=1}^n (y_i - \hat{y}_i)^2$. Se elevan al cuadrado por dos razones: para que errores positivos y negativos no se cancelen entre sí, y para penalizar más los errores grandes que los pequeños (un error de 10 pesa 100; uno de 2 pesa 4). Guarda este detalle: explica por qué los outliers distorsionan tanto este algoritmo, como veremos en los errores comunes.

Ejemplo numérico (el caso inmobiliario del PDF, con números): supón que el modelo aprendió

$$\text{precio} = 30.000 + 1.500 \cdot m^2 + 8.000 \cdot \text{habitaciones} - 900 \cdot \text{antigüedad}$$

Una casa de 80 m², 3 habitaciones y 10 años: $30.000 + 1.500(80) + 8.000(3) - 900(10) = 30.000 + 120.000 + 24.000 - 9.000 = 165.000$ dólares. Y la lectura de coeficientes es directa: cada m² adicional suma \$1.500 al precio; cada año de antigüedad resta \$900. En Fibromin: $\text{vida útil restante} = \beta_0 - \beta_1 \cdot \text{vibración} - \beta_2 \cdot \text{temperatura}$, donde los signos negativos te dicen que más vibración y más temperatura acortan la vida del equipo, y la magnitud te dice cuánto.

Su gran ventaja es exactamente esa: la interpretabilidad. Los coeficientes muestran qué variables tienen mayor impacto, en qué dirección y en cuánto. Por eso sigue dominando en finanzas, seguros, salud y análisis económico, donde explicar la predicción importa tanto como acertar.

Sus limitaciones: supone que las relaciones entre variables son aproximadamente **lineales** y que las características tienen cierto grado de **independencia** entre sí (cuando dos features están muy correlacionadas —**multicolinealidad**— los coeficientes se vuelven inestables y pierden su lectura limpia). Frente a fenómenos con patrones complejos o altamente dinámicos, su desempeño cae considerablemente. Aun así, sigue siendo una excelente **línea de base** (*baseline*) para iniciar cualquier proyecto analítico: primero la recta, después la artillería.

Árboles de decisión (2 de 3)

Los **árboles de decisión** (*decision trees*) operan con una lógica completamente distinta: en lugar de una ecuación global, construyen una **secuencia jerárquica de reglas** del tipo "si

ocurre esto, sigue por este camino". Cada **nodo** divide los datos según una condición; las ramas terminan en **hojas** que contienen la predicción final.

La figura del PDF es el clásico árbol de **supervivencia de los pasajeros del Titanic**, y vale la pena leerla con cuidado porque enseña a interpretar cualquier árbol. El nodo raíz pregunta por el **género**: si es **mujer**, la hoja dice "Sobrevivió, 0.73; 36%" — es decir, el 36% de los pasajeros cae en esta rama y su probabilidad de supervivencia fue 73%. Si es **hombre**, el árbol pregunta por la **edad**: si la edad es mayor a 9.5 años, "Murió, 0.17; 61%" (el 61% de los pasajeros eran hombres mayores de 9.5 años, y solo el 17% de ellos sobrevivió). Si era un niño (edad ≤ 9.5), el árbol pregunta cuántos **hermanos/cónyuges a bordo** tenía: con 3 o más, "Murió, 0.02; 2%" (familias numerosas: 2% de los pasajeros, 2% de supervivencia); con menos de 3, "Sobrevivió, 0.89; 2%". En cada hoja, el primer número es la **probabilidad de supervivencia** y el segundo el **porcentaje de pasajeros** que cae allí. Tres preguntas encadenadas reconstruyen el "mujeres y niños primero" sin que nadie se lo dijera al modelo: lo dedujo de los datos.

Ventajas: son **extremadamente intuitivos para usuarios de negocio**, porque se parecen a los diagramas de decisión que las organizaciones usan hace décadas. Además, manejan variables **numéricas y categóricas**, capturan **relaciones no lineales** y toleran ciertos niveles de **datos faltantes**. Por eso se usan en *scoring* crediticio, segmentación de clientes, clasificación de riesgo y diagnóstico médico. Su interpretabilidad es especialmente valiosa en **industrias reguladas**, donde hay que justificar por qué el modelo decidió lo que decidió (¿por qué le negaste el crédito a esta persona? Porque ingresos $< X$ y morosidad previa = sí — auditable línea por línea).

Problemas: los árboles simples **tienden a sobreajustarse con facilidad**: si los dejas crecer sin límite, generan divisiones tan específicas que terminan memorizando detalles irrelevantes del entrenamiento ("si el cliente tiene 37 años, vive en tal comuna y compró un martes, entonces..."). Además son **inestables**: pequeños cambios en los datos pueden producir árboles completamente distintos, lo que reduce robustez. Estas dos debilidades —sobreajuste e inestabilidad— son precisamente lo que motiva a la siguiente familia.

Métodos de conjunto (3 de 3)

Los **métodos de conjunto** (*Ensemble Methods*) encarnan una de las ideas más poderosas del ML moderno: **combinar múltiples modelos débiles para construir un modelo fuerte**. En vez de apostar todo a un único algoritmo, integran muchas predicciones para obtener resultados más estables y precisos. La intuición es la sabiduría de las multitudes: un jurado de 500 árboles mediocres, si sus errores son independientes, le gana a un solo árbol brillante, porque los errores individuales se cancelan al promediar.

Dentro de la familia hay **dos grandes estrategias**, y distinguirlas es pregunta segura de examen:

Bagging (*Bootstrap Aggregating*): entrena múltiples modelos **independientes y en paralelo**, cada uno con un subconjunto distinto de los datos (muestras aleatorias con reemplazo, llamadas *bootstrap*), y luego **combina sus resultados por promedio** (regresión) o **votación** (clasificación). El ejemplo más conocido es **Random Forest** (bosque aleatorio), que además de submuestrear filas, le da a cada árbol un subconjunto aleatorio

de *variables* en cada división, forzando diversidad entre los árboles. Bagging ataca principalmente la **inestabilidad** (la varianza) de los árboles individuales. Analogía: encargas el mismo diagnóstico a 500 médicos que no se hablan entre sí y te quedas con la opinión mayoritaria.

Boosting: construye modelos **de manera secuencial**, donde cada nuevo modelo **intenta corregir los errores cometidos por el anterior**, concentrándose en los casos que los previos clasificaron mal. A esta categoría pertenecen **Gradient Boosting** (la formulación general: cada modelo nuevo se ajusta a los residuos del conjunto acumulado), **XGBoost** (implementación optimizada y regularizada, durante años la reina de las competencias) y **LightGBM** (variante más rápida y liviana para grandes volúmenes). Analogía: un solo alumno que da un ensayo, revisa exactamente qué preguntas falló, estudia esas, da otro ensayo, y repite — cada iteración pule los errores de la anterior.

Ventaja: un **enorme poder predictivo**. Durante años estos métodos han dominado las competencias de ciencia de datos y hoy son el estándar para problemas empresariales complejos con datos tabulares.

Costo: la precisión extra llega con **menor interpretabilidad**. Sumar cientos de árboles convierte al modelo en una "**caja negra**" (*black box*) difícil de explicar a usuarios no técnicos: ya no hay una rama que puedas mostrar en una auditoría. Por eso la selección del algoritmo no depende solo de la precisión: también pesan **la regulación, la facilidad de auditoría, la capacidad técnica del equipo y el nivel de confianza** que la organización exige. *Nota breve*: existen técnicas post-hoc de explicabilidad (como SHAP) que mitigan parcialmente la caja negra, pero el curso no las cubre.

Modelos para datos no estructurados

Todo lo anterior asume **datos estructurados**: tablas ordenadas de filas y columnas. Pero una enorme cantidad de información empresarial vive en **formatos no estructurados**: documentos, imágenes, videos, audio, texto libre. (Conexión con el módulo de Big Data: esto es la **Variedad**, la tercera V.) Cuando el problema involucra este tipo de datos, entramos al terreno del **aprendizaje profundo**.

Aprendizaje profundo (Deep Learning)

El **aprendizaje profundo** (*deep learning*) usa **redes neuronales artificiales** (*artificial neural networks*) inspiradas conceptualmente en el cerebro humano: múltiples **capas interconectadas** de unidades simples ("neuronas") cuyas conexiones tienen pesos ajustables. Lo de "profundo" es literal: muchas capas apiladas, donde cada capa aprende **representaciones cada vez más abstractas** de los datos — en una imagen, las primeras capas detectan bordes, las intermedias texturas y formas, las finales objetos completos.

La diferencia clave con los modelos tradicionales —y la idea más examinable de esta sección—: en los modelos clásicos, **un especialista debe definir manualmente qué variables son relevantes** (la ingeniería de características del Paso 2); las redes profundas

aprenden automáticamente los patrones relevantes directamente desde imágenes, texto o audio bruto. El feature engineering deja de ser manual.

¿Por qué brillan justo ahí? Por la **dimensionalidad**. El recuadro del PDF lo cuantifica: una tabla empresarial tiene decenas o cientos de columnas, pero **una imagen pequeña representa cientos de miles de valores de píxel** (una foto de 300×300 píxeles a color son $300 \times 300 \times 3$ canales = 270.000 números). Ningún humano puede diseñar a mano variables útiles sobre 270.000 entradas; las redes profundas detectan relaciones útiles dentro de esa complejidad y las convierten en predicciones o clasificaciones precisas.

Gracias a esto, el deep learning impulsó los grandes avances en reconocimiento de voz, análisis de imágenes, traducción automática y **sistemas generativos**. Sus **desafíos**, también examinables: exige **grandes volúmenes de datos**, **capacidad computacional considerable** (GPUs), **tiempos de entrenamiento largos**, y es **menos interpretable** que los modelos tradicionales — la caja negra llevada al extremo: explicar exactamente por qué la red decidió algo es genuinamente difícil.

Procesamiento del Lenguaje Natural (PLN)

El **procesamiento del lenguaje natural** (*Natural Language Processing, NLP*) busca que las máquinas **comprendan, clasifiquen y generen lenguaje humano**. La disciplina dio un salto enorme gracias a una arquitectura moderna: los **transformadores** (*transformers*), que hoy sustentan los asistentes conversacionales, los sistemas generativos y los modelos de lenguaje avanzados (la familia de la que descende, dicho sea de paso, el asistente que redactó esta explicación). La idea central del transformer es el **mecanismo de atención**: cada palabra de una secuencia "mira" a todas las demás para decidir cuáles son relevantes para interpretarla, y todo se procesa en paralelo — eso permitió entrenar con volúmenes de texto antes impensables.

¿Por qué le importa al negocio? Porque **gran parte de la información corporativa está en lenguaje natural**: correos electrónicos, contratos, comentarios de clientes, publicaciones en redes sociales, tickets de soporte. Ahí hay señales valiosísimas sobre comportamiento, satisfacción, riesgos y oportunidades. El gran aporte del PLN es **transformar ese lenguaje humano en variables analíticas accionables**. Las aplicaciones que lista el PDF: análisis de sentimiento, clasificación automática de documentos, traducción automática, chatbots, búsqueda semántica y generación de texto. En Fibromin: los reportes de mantención escritos a mano alzada por los técnicos ("se escucha un golpeteo al partir en frío") son oro no estructurado; un clasificador de PLN puede convertirlos en una feature más del modelo predictivo.

Visión por computador

La **visión por computador** (*computer vision*) persigue el objetivo equivalente en el dominio visual: que las máquinas **interpreten imágenes y video** de manera similar a como lo hacen los humanos. Hoy estos sistemas pueden reconocer objetos, detectar anomalías, identificar rostros, segmentar escenas y extraer texto desde documentos visuales (OCR).

Muchas de estas capacidades se basan en **redes neuronales convolucionales** (*Convolutional Neural Networks, CNNs*), especializadas en detectar **patrones espaciales** dentro de imágenes. La intuición de la convolución: en vez de mirar los 270.000 píxeles de golpe, la red desliza pequeños **filtros** sobre la imagen que detectan patrones locales (un borde, una esquina, una textura), y las capas superiores combinan esos patrones locales en estructuras mayores. Es como reconocer una cara: primero ojos, nariz y boca por separado, luego su disposición conjunta.

Las aplicaciones empresariales del PDF: en **industria**, control automático de calidad; en **logística**, lectura de etiquetas y seguimiento de paquetes; en **salud**, apoyo diagnóstico con imágenes médicas; en **retail**, monitoreo de inventario y análisis del comportamiento de clientes en tienda. Para Fibromin: termografías de motores e inspección visual automatizada de componentes son visión por computador aplicada a mantenimiento (y recuerda el módulo anterior: alto impacto, pero viabilidad menor que el predictivo con sensores que ya tienes — no es caso campeón hoy). La capacidad de **convertir imágenes en información estructurada** ha transformado industrias completas y sigue expandiéndose con más poder de cómputo y más datos visuales disponibles.

Aprendizaje por refuerzo

El **aprendizaje por refuerzo** (*reinforcement learning*) es una lógica distinta al aprendizaje supervisado — fíjate que el PDF lo incluye aquí como contraste de paradigma. El modelo (el **agente**) **no aprende de ejemplos históricos etiquetados**: **interactúa continuamente con un entorno dinámico**, prueba **acciones** y recibe **recompensas o castigos** según el resultado. Con el tiempo aprende **estrategias que maximizan la recompensa acumulada** y se adapta al comportamiento del entorno.

La diferencia conceptual clave, examinable: en supervisado existe una respuesta correcta predefinida para cada ejemplo (el ground truth); en refuerzo **no hay respuesta correcta predefinida** — el sistema debe **descubrir por sí mismo qué secuencia de acciones produce mejores resultados a largo plazo**. Analogía: entrenar a un perro. No le muestras un manual etiquetado de "conductas correctas"; le das premios cuando acierta y nada cuando no, y él infiere la política. O aprender a andar en bicicleta: nadie te entrega ejemplos etiquetados de equilibrio; te caes (castigo), avanzas (recompensa), y tu cerebro optimiza.

Sus éxitos: videojuegos, robótica, optimización industrial y sistemas de recomendación personalizados. Su fortaleza aparece en problemas donde **las decisiones afectan estados futuros** y el sistema necesita **adaptarse continuamente** — decidir hoy cambia el escenario de mañana, algo que el supervisado clásico no modela.

Síntesis de algoritmos: el menú va desde modelos interpretables (regresión lineal, árboles) hasta métodos complejos (conjuntos, deep learning, PLN, visión, refuerzo). El desafío nunca es "usar el más potente", sino **elegir la solución adecuada según el problema, los datos disponibles, la necesidad de explicación y el contexto operativo** de la organización.

Evaluación de modelos y errores comunes

La evaluación es el paso crítico donde se **comparan los valores predichos** por el sistema **con las etiquetas reales o "verdad básica"** (*ground truth*) **del conjunto de prueba**. El propósito: determinar si el modelo **generaliza** ante datos que no vio durante el entrenamiento. Fíjate en la arquitectura del argumento: por eso reservamos el test set en el Paso 3 — sin esa reserva, esta etapa no tendría contra qué medir honestamente.

Evaluación de modelos de regresión

Cuando se predicen valores numéricos continuos, las métricas se centran en la **magnitud del error**.

RMSE (*Root Mean Squared Error*, raíz del error cuadrático medio):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Desglose: y_i es el valor real del caso i ; $\hat{y}_i$ es la predicción del modelo; $(y_i - \hat{y}_i)$ es el error individual; se eleva al **cuadrado** para que los errores positivos y negativos no se cancelen y para castigar más los errores grandes; se **promedia** sobre los n casos (eso, sin la raíz, es el **MSE**, *Mean Squared Error*); y la **raíz** final devuelve el resultado a la escala original de los datos.

Esa última propiedad es su gran ventaja práctica: **se expresa en las mismas unidades que lo que predices**. El ejemplo del PDF: si un modelo predice precios de casas en dólares y tiene RMSE de 256.6, significa que, en promedio, sus predicciones se desvían **256.6 dólares** del valor real. Mini-ejemplo numérico con tres casas: errores de +200, -300 y +250 dólares → cuadrados: 40.000, 90.000, 62.500 → promedio (MSE): 64.167 → raíz (RMSE): ≈ 253 dólares. Nota cómo el error de 300 pesa desproporcionadamente más: el cuadrado amplifica los errores grandes, detalle que reaparecerá con los outliers.

Estadístico R-cuadrado (R^2 , *coeficiente de determinación*): mide **qué proporción de la variación total de los datos es explicada por el modelo**. Su fórmula:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

El numerador es el error que comete **tu modelo** (suma de residuos al cuadrado); el denominador es el error que cometería el **modelo más tonto posible**: predecir siempre el promedio $\bar{y}$. R^2 responde entonces: ¿cuánto mejor que "predecir el promedio" es mi modelo? Oscila entre **0 y 1**: un valor de 1 indicaría que el modelo explica **toda** la varianza; 0, que no aporta nada sobre el promedio. Un $R^2 = 0{,}85$ (el del caso LTV) se lee: "el modelo explica el 85% de la variación de los datos". Es la métrica de **ajuste** a la estructura de los datos. *Advertencia breve*: un R^2 alto en entrenamiento no garantiza buen desempeño fuera de muestra ni implica causalidad; por eso se mide sobre el test set.

Análisis de residuos: más allá de los números resumen, mirar los **residuos** —la diferencia entre valor real y predicho, $e_i = y_i - \hat{y}_i$ — entrega información visual valiosa. En el **diagrama de residuos** (residuos en el eje vertical vs. valores predichos en el horizontal), el patrón ideal es una **nube de puntos dispersos aleatoriamente** alrededor de cero, sin

estructura. Eso confirma el supuesto de **linealidad** (no quedó ninguna curva sin capturar) y el de **dispersión uniforme de los errores en todos los rangos de predicción** — propiedad cuyo nombre técnico correcto es **homocedasticidad** (*homoscedasticity*). ⚠ **Ojo: el PDF llama a esto "heteroscedasticidad", y es un error del material** — heterocedasticidad es exactamente lo contrario (la dispersión que cambia, típicamente en forma de embudo). Detalle completo en las Notas de discrepancia. Los patrones en los residuos son diagnósticos: una **curva** delata no linealidad; un **embudo** delata heterocedasticidad; unos pocos puntos lejanos delatan **outliers** — exactamente el diagnóstico que salvará al caso LTV.

Evaluación de modelos de clasificación

Para categorías, la herramienta madre es la **matriz de confusión** (*confusion matrix*): una tabla que cruza lo que el modelo predijo contra lo que realmente pasó, en cuatro cuadrantes:

	Real: Positivo	Real: Negativo
Predicho: Positivo	Verdadero positivo (TP)	Falso positivo (FP) — Error Tipo I
Predicho: Negativo	Falso negativo (FN) — Error Tipo II	Verdadero negativo (TN)

Para memorizar los errores: el **falso positivo** es la **falsa alarma** (gritar "¡fuego!" sin incendio); el **falso negativo** es la **omisión** (haber incendio y que la alarma calle). En el lenguaje de un juicio: Error Tipo I es condenar a un inocente; Tipo II, absolver a un culpable.

Trabajemos las cuatro métricas derivadas con un ejemplo numérico de Fibromin: 1.000 lecturas de equipos, de las cuales **50 corresponden a fallas reales**. Tu modelo marca 60 alertas: 40 son fallas verdaderas (TP = 40), 20 son falsas alarmas (FP = 20); se le escapan 10 fallas (FN = 10) y acierta los 930 equipos sanos (TN = 930).

1. Exactitud (*Accuracy*): la frecuencia total de aciertos.

$$\$Exactitud = \frac{TP + TN}{TP + TN + FP + FN} = \frac{40 + 930}{1000} = 97\%\$$$

Suena excelente, pero **solo funciona para problemas equilibrados**. La trampa, con el ejemplo del PDF: si el fraude ocurre en el **0.1%** de las transacciones, un modelo que prediga *siempre* "no fraude" tendrá **99.9% de exactitud y será completamente inútil**, porque no detecta ni un caso real. En nuestro ejemplo Fibromin pasa lo mismo en pequeño: un "modelo" que diga "nunca falla nada" logra 95% de exactitud (950/1000) sin detectar una sola falla. A esto se le llama la *paradoja de la exactitud*, y es la razón de existir de las tres métricas siguientes.

2. Precisión (*Precision*): la **calidad de las predicciones positivas** — de todo lo que etiquetaste como positivo, ¿cuánto lo era realmente?

$$\$Precisión = \frac{TP}{TP + FP} = \frac{40}{60} = 66\%\$$$

De cada 3 alertas del modelo, 2 son fallas reales y 1 es falsa alarma. Se **prioriza cuando el costo de un falso positivo es alto**: filtros de spam (mandar un correo importante a spam es caro), campañas con presupuesto limitado.

3. Recuperación (Recall, también llamada sensibilidad): la **capacidad de encontrar todos los positivos reales** — de las fallas que existían, ¿cuántas cazaste?

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{40}{50} = 80\%$$

Detectaste 8 de cada 10 fallas reales; 2 se te escaparon. Es **vital cuando el costo de omitir un positivo (falso negativo) es crítico**: detección de enfermedades, de fraudes — y de fallas de equipos: en Fibromin, el recall es probablemente tu métrica reina, porque la falla no detectada es la parada catastrófica.

4. Puntuación F1 (F1 score): la **media armónica** entre precisión y recall.

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} = 2 \cdot \frac{0.667 \cdot 0.80}{0.667 + 0.80} \approx 0.73$$

¿Por qué media **armónica** y no aritmética? Porque la armónica **penaliza fuertemente si una de las dos métricas es muy baja**, mientras la aritmética lo esconde. Demostración numérica: con precisión 0.9 y recall 0.1, la media aritmética da un respetable 0.5; la armónica da $2 \cdot \frac{0.9 \cdot 0.1}{0.9 + 0.1} = 0.18$, delatando el desastre. Un F1 alto exige que **ambas** métricas sean decentes a la vez — por eso es una medida robusta del **equilibrio** del modelo.

ROC-AUC (mencionada en el Paso 5 del flujo): la **curva ROC** grafica la tasa de verdaderos positivos contra la tasa de falsos positivos para **todos los umbrales de decisión posibles** (la mayoría de los clasificadores no dice "falla/no falla" sino "probabilidad de falla = 0.73", y tú eliges desde qué probabilidad disparas la alerta). El **AUC (Area Under the Curve)** resume esa curva en un número: **0.5 equivale al azar** (lanzar una moneda), **1.0 a la separación perfecta** de clases. Su gracia: permite comparar modelos sin casarse con un umbral específico. Y el umbral mismo es una palanca de negocio: bajarlo sube el recall a costa de la precisión (más alertas, más falsas alarmas), subirlo hace lo inverso. Elegir el umbral es elegir qué error prefieres cometer.

Errores comunes y cómo evitarlos

El éxito de una iniciativa de IA no depende solo de las métricas, sino de evitar cinco **trampas estratégicas**:

1. Usar ML cuando no es necesario. Antes de implementar algoritmos complejos, establece una **línea de base simple (baseline)** con reglas lógicas o heurísticas, y solo avanza hacia el aprendizaje automático **si demuestra superar significativamente esa referencia**. La regla "cliente en riesgo = no compra hace 90 días" de NovaMarket, o "alerta si temperatura > 85°C" en Fibromin, son baselines legítimas: gratis, explicables, mantenibles. Si tu Random Forest le gana por dos puntos porcentuales, quizás no paga el costo de complejidad. El ML se justifica por su *delta* sobre lo simple, no por su sofisticación.

2. Sobreajuste (*overfitting*). Ocurre cuando el modelo es tan complejo que **memoriza el ruido** del entrenamiento y falla con datos nuevos. El objetivo es encontrar el **"punto óptimo"** (*sweet spot*): la zona donde **el error de entrenamiento y el de prueba son bajos y similares**. La señal de alarma es la brecha: 98% en entrenamiento y 70% en prueba no es un buen modelo, es un memorión. El espectro completo: un modelo demasiado simple falla en ambos conjuntos (subajuste); uno demasiado complejo brilla en entrenamiento y se estrella en prueba (sobreajuste); el sweet spot está en medio. *Nota breve:* las mitigaciones habituales —más datos, modelos más simples, regularización, poda de árboles— exceden el alcance del curso.

3. La maldición de la dimensionalidad (*curse of dimensionality*). Incluir demasiadas variables —especialmente **categoricas con muchos valores únicos**, como códigos postales— hace que los datos se vuelvan **"dispersos"** (*sparse*): el espacio de combinaciones posibles crece exponencialmente con cada variable, y tus mismos 10.000 registros que poblaban bien un espacio de 5 dimensiones quedan como granos de arena perdidos en uno de 500. Los algoritmos no encuentran patrones reales en ese vacío. La recomendación: **reducir la dimensionalidad** eliminando características redundantes o **combinándolas** — el ejemplo del PDF: fusionar "precio" y "tamaño" en un solo atributo de **"precio por metro cuadrado"**. En Fibromin: en vez de 200 sensores crudos como 200 columnas, features agregadas como "promedio de vibración última semana" o "delta de temperatura vs. línea base del equipo". Es feature engineering al servicio de la parsimonia.

4. Ignorar los valores atípicos (*outliers*). Puntos de datos extremos pueden **sesgar drásticamente una línea de regresión**, invalidando el ajuste para el resto de las observaciones. ¿Recuerdas que OLS minimiza errores *al cuadrado*? Un solo punto absurdamente lejano domina esa suma y arrastra la recta hacia él. Es fundamental **detectarlos y decidir** si se eliminan para mejorar la precisión general. El matiz importante: decidir, no borrar en automático — a veces el outlier es la señal (en detección de fraude, los outliers son justamente lo que buscas; en Fibromin, una lectura extrema puede ser un sensor malo... o la falla incubándose).

5. No detenerse con suficientes datos. Los modelos suelen alcanzar una **meseta de precisión** donde añadir más datos **ya no mejora el rendimiento** significativamente, pero **sí dispara los costos de etiquetado**. Rendimientos decrecientes puros: pasar de 1.000 a 10.000 ejemplos puede regalarte 15 puntos de precisión; de 100.000 a 1.000.000, medio punto, pagando diez veces más por etiquetar. La defensa: tener un **objetivo de precisión claro definido de antemano**, y parar al alcanzarlo.

Caso de aplicación: modelado del valor de vida del cliente (LTV)

El PDF cierra la sección con un caso que integra todo. El **LTV** (*Lifetime Value*, valor de vida del cliente) busca **predecir cuánto dinero gastará un cliente en el futuro**. Sigue la cadena de decisiones:

La etiqueta es **numérica** (dinero) → es un problema de **regresión**. Los datos de comportamiento son **complejos** (no lineales) → el equipo elige un **método de conjunto, Boosting**, para maximizar poder predictivo. Dividen en entrenamiento y prueba, y evalúan con **RMSE**... que sale **muy alto**. En vez de rendirse o cambiar de algoritmo a ciegas,

analizan los residuos — y el diagnóstico visual revela **valores atípicos**: clientes corporativos que compran volúmenes masivos, mezclados con personas naturales. La medicina: **separar esos segmentos y reentrenar**. Resultado: $R^2 = 0,85$ — el modelo explica el 85% de la varianza, suficiente para que el departamento de finanzas haga **proyecciones de ingresos más precisas**.

La moraleja del caso, en palabras del PDF: evaluar un modelo no es solo revisar si "acierta", sino **comprender qué tipo de error comete, cuánto impacta ese error en el negocio y si el modelo funciona con datos nuevos**. Las métricas no solo califican: **diagnostican**. El RMSE alto fue el síntoma; los residuos, la radiografía; los outliers, el diagnóstico; la segmentación, el tratamiento.

Caso integrador: predicción de clientes en riesgo en NovaMarket

Contexto. NovaMarket es un e-commerce de productos para el hogar, tecnología y bienestar, que creció rápido con campañas digitales y descuentos personalizados. El equipo comercial detecta el problema: muchos clientes compran una o dos veces y desaparecen; el **costo de adquisición** sube y la rentabilidad se resiente (la economía de fondo: adquirir un cliente nuevo cuesta varias veces más que retener uno existente, así que la fuga silenciosa erosiona el negocio por los dos lados). La gerencia pide un modelo que **anticipe qué clientes tienen mayor probabilidad de abandonar en los próximos tres meses** — churn, el clásico.

El problema con los datos. El equipo recopila historia: número de compras, monto promedio gastado, frecuencia de navegación, uso de cupones, reclamos registrados, tiempo desde la última compra y participación en campañas de email. Y choca con la realidad del Paso 1: **registros incompletos, formatos distintos y comportamientos atípicos** (empresas comprando volúmenes grandes en fechas específicas — los mismos outliers del caso LTV). La lección temprana: **no basta tener muchos datos; hay que organizarlos** para que el modelo pueda procesarlos.

La discusión clave. ¿Modelo complejo de entrada, o **línea de base simple** primero? La pregunta central del caso: **¿realmente se necesita machine learning o basta una regla de negocio bien diseñada?** El equipo toma la decisión correcta —la primera decisión crítica, dice el PDF—: **no asumir** que lo complejo es mejor, y comparar contra la regla "cliente en riesgo = no compra hace más de 90 días".

Los 7 pasos aplicados (nota cómo el caso es un espejo exacto del flujo de la primera sección):

1. **Recopilar y organizar datos:** cada cliente una fila, cada variable relevante una columna (tidy data en acción).
2. **Definir características y etiqueta:** features = frecuencia de compra, reclamos, uso de cupones...; label = **abandono / no abandono** (clasificación binaria).
3. **Dividir los datos** en entrenamiento y prueba.

4. **Probar algoritmos:** la regla simple vs. un árbol de decisión vs. métodos de conjunto (Random Forest, Boosting).
5. **Evaluar** sin quedarse solo con la exactitud: precisión, recuperación y F1.
6. **Implementar:** el modelo se integra al sistema comercial y asigna **semanalmente una puntuación de riesgo** a cada cliente (inferencia por lotes).
7. **Mantener y reentrenar** cuando los patrones cambien (drift).

Por qué la baseline no bastó. La regla de 90 días detectaba algunos casos, pero **dejaba fuera a clientes que todavía navegaban en la plataforma pero ya mostraban señales de pérdida de interés** — patrones sutiles, multivariables, que una regla de un solo umbral no captura. Ahí el ML sí justifica su delta. Escalaron con criterio: primero un **árbol de decisión** por su facilidad de explicación, luego **Random Forest y Boosting** para exprimir precisión.

Por qué la exactitud no bastó. Un modelo podía "parecer correcto" sin detectar realmente a los clientes en riesgo (la paradoja de la exactitud: el churn es minoría, así que predecir "nadie se va" da exactitud alta e información nula). Por eso revisaron **precisión, recall y F1**. Y la elección final fue un **equilibrio**: detectar a los que se van (recall) sin ahogar al equipo en **falsas alarmas** (precisión). El análisis de costos asimétricos: un falso positivo cuesta un descuento innecesario a un cliente que igual se quedaba; un falso negativo cuesta **el cliente completo y su LTV futuro**. El FN es más caro, lo que inclina la balanza hacia el recall — pero con un piso de precisión, porque regalar descuentos masivos a quien no los necesita también destruye margen. F1 es justamente la métrica de ese equilibrio.

Del output al outcome. Con la puntuación semanal (el *output*), marketing diseñó **campañas específicas: descuentos personalizados, mensajes de reactivación y beneficios por recompra** (el *outcome*: retención y rentabilidad). El modelo no retiene clientes; habilita las acciones que los retienen.

Conclusión del caso (literal del curso, examinable): un proyecto de IA no se resuelve solo eligiendo un algoritmo, sino **tomando decisiones coherentes con el problema de negocio, los datos disponibles y el tipo de error que la empresa puede asumir**. Y la implementación **no es el cierre del proceso**: el modelo debe monitorearse y actualizarse para seguir siendo útil en un entorno cambiante.

Guía para las actividades de la unidad

El PDF incluye varias instancias participativas. No te doy las respuestas hechas (son tuyas), pero sí el ángulo que cada pregunta está evaluando:

- **Foro 1 (proceso y algoritmos):** la etapa "más crítica" admite defender varias (datos, evaluación, mantención) — lo evaluado es que **justifiques con el costo del error en esa etapa**. Precisión vs. explicabilidad: no hay respuesta única; depende de regulación y criticidad ética (módulo anterior). Riesgos de no monitorear: drift y decisiones tomadas sobre un modelo vencido.

- **Foro 2 (evaluación):** exactitud alta $\neq$ utilidad $\rightarrow$ paradoja de la exactitud con clases desbalanceadas. Bueno en entrenamiento, malo en prueba $\rightarrow$ sobreajuste: simplificar el modelo, conseguir más datos, revisar leakage.
- **Caso NovaMarket (preguntas):** riesgos de saltarse la baseline $\rightarrow$ costo y complejidad sin delta demostrado; por qué accuracy no basta $\rightarrow$ churn es clase minoritaria; qué error es más caro $\rightarrow$ FN (pierdes el cliente y su LTV) vs. FP (un descuento de más); mantener vigente el modelo $\rightarrow$ monitoreo, reentrenamiento, MLOps.
- **Ejercicio práctico (retail / programa de fidelización):** es NovaMarket con otro nombre — replica la estructura: clasificación binaria, label "abandona el programa en 3 meses sí/no", features de comportamiento transaccional, división temporal de datos, árbol primero por interpretabilidad, **recall** como métrica prioritaria (el enunciado pide "identificar la mayor cantidad posible de clientes que realmente abandonarán" — eso es la definición de recall), y mantención por drift.
- **Foro colaborativo (transporte / horas punta):** problema de **regresión** (cantidad de vehículos = número continuo). El alto margen de error puede venir de los datos (features insuficientes: clima, eventos, estacionalidad), del modelo (subajuste a patrones no lineales) o de ambos. RMSE te dice cuántos vehículos te desvías en promedio (interpretable en unidades de flota); R^2 cuánta de la variación de demanda explicas.

Material complementario: los canales **Dot CSV** (divulgación técnica en español sobre deep learning y redes neuronales) y **StatQuest con Josh Starmer** (en inglés con subtítulos; visual y claro desde regresión lineal hasta redes neuronales — recomendación personal: sus videos de Random Forest y de ROC son oro para esta unidad). **Bibliografía:** Zwingmann, T. (2024), *Inteligencia empresarial basada en IA* (O'Reilly); Calvo, J. & Escapa, C. (2025), *The AI Driven Business* (Libros de Cabecera).

Notas de discrepancia

1. "Heteroscedasticidad" definida al revés (error del material). El PDF dice que un diagrama de residuos ideal confirma supuestos de "linealidad y **heteroscedasticidad** (dispersión uniforme de errores en todos los rangos de predicción)". La definición entre paréntesis corresponde a la **homocedasticidad**: varianza constante de los residuos, que es la propiedad *deseable* y el supuesto del modelo lineal. La **heterocedasticidad** es lo contrario —dispersión no uniforme, el clásico embudo— y es un *problema*, no algo que quieras confirmar. Para el examen: si la pregunta replica la frase del curso, la opción esperada puede usar la palabra del PDF; pero si la pregunta es conceptual ("¿cómo se llama la dispersión uniforme de los residuos?"), la respuesta correcta es homocedasticidad. Domina ambas y sabrás responder cualquiera de las dos variantes.

2. RMSE no es "el error cuadrático medio". El PDF traduce RMSE como "Error cuadrático medio", pero ese nombre corresponde al **MSE** (*Mean Squared Error*). El RMSE es la **raíz** del MSE (*Root MSE*) — y justamente esa raíz es lo que le da su ventaja de expresarse en la escala original. Detalle menor, pero en una pregunta de alternativas con MSE y RMSE como distractores mutuos, la diferencia es la raíz.

3. El título dice "descriptivo"; el contenido es predictivo. En la taxonomía estándar de analítica (descriptiva → diagnóstica → predictiva → prescriptiva), el análisis **descriptivo** responde "¿qué pasó?" (dashboards, KPIs, BI), mientras que predecir churn, LTV o fallas futuras es análisis **predictivo** ("¿qué va a pasar?"). Todo el contenido de esta unidad —aprendizaje supervisado, predicciones, inferencia— es predictivo. Responde según el contexto de cada pregunta, pero no dejes que el título te haga clasificar el ML supervisado como herramienta descriptiva en una pregunta de taxonomía.

4. "Drift" agrupado. El PDF habla de "**drift** o deriva de datos" como fenómeno único. Como vimos en el módulo anterior, la literatura distingue **data drift** (cambia la distribución de las entradas) de **concept drift** (cambia la relación entrada→salida). Para esta unidad, "drift" a secas es la respuesta esperada; el matiz te da ventaja en preguntas finas y en la vida real con Fibromin.

Glosario

Término	Inglés	Definición en una línea
Aprendizaje automático supervisado	Supervised machine learning	Paradigma que aprende de ejemplos históricos etiquetados para predecir casos nuevos.
Característica	Feature	Variable de entrada que el modelo usa como señal predictiva.
Etiqueta	Label	Variable objetivo que el modelo intenta predecir.
Verdad básica	Ground truth	Respuesta correcta verificada contra la que se entrena y evalúa el modelo.
Regresión	Regression	Problema cuya salida es un valor numérico continuo.
Clasificación	Classification	Problema cuya salida es una categoría discreta.
Datos ordenados	Tidy data	Una fila por observación, una columna por variable, un valor por celda.
Conjunto de entrenamiento	Training set	Porción de datos con la que el algoritmo aprende los patrones.
Conjunto de validación	Validation set	Porción para ajustar hiperparámetros sin contaminar la prueba.
Conjunto de prueba	Test set	Porción reservada para evaluar el desempeño final con datos no vistos.

Fuga de datos	Data leakage	Contaminación del entrenamiento con información del futuro o del test ("ver el futuro").
Algoritmo	Algorithm	Procedimiento matemático utilizado para aprender.
Entrenamiento	Training	Proceso de ajuste de parámetros a partir de datos históricos.
Modelo	Model	Resultado final aprendido tras el entrenamiento.
Parámetro	Parameter	Valor interno que el modelo aprende (p. ej., los coeficientes β).
Hiperparámetro	Hyperparameter	Configuración fijada antes de entrenar (p. ej., profundidad del árbol).
Función de pérdida	Loss function	Medida del error que el entrenamiento busca minimizar.
Inferencia	Inference	Fase en que el modelo entrenado genera predicciones sobre datos nuevos sin seguir aprendiendo.
Procesamiento por lotes	Batch processing	Inferencia ejecutada periódicamente sobre grupos de casos.
API	Application Programming Interface	Interfaz que permite consultar el modelo en tiempo real desde otros sistemas.
Sobreajuste	Overfitting	El modelo memoriza el ruido del entrenamiento y falla al generalizar.
Subajuste	Underfitting	El modelo es demasiado simple y falla incluso en el entrenamiento.
Punto óptimo	Sweet spot	Zona donde el error de entrenamiento y el de prueba son bajos y similares.
Regresión lineal	Linear regression	Modelo que ajusta una relación lineal entre features y una salida numérica.
Coeficiente	Coefficient	Peso que indica cuánto y en qué dirección influye cada variable.
Intercepto	Intercept	Valor base de la predicción cuando todas las features valen cero.

Mínimos cuadrados ordinarios	Ordinary Least Squares (OLS)	Método que estima los coeficientes minimizando la suma de residuos al cuadrado.
Multicolinealidad	Multicollinearity	Correlación alta entre features que vuelve inestables los coeficientes.
Línea de base	Baseline	Solución simple de referencia que el ML debe superar para justificarse.
Árbol de decisión	Decision tree	Modelo de reglas jerárquicas si-entonces que divide los datos en nodos.
Nodo / hoja	Node / leaf	Punto de división del árbol / punto terminal con la predicción.
Métodos de conjunto	Ensemble methods	Combinación de varios modelos débiles para formar uno fuerte.
Bagging	Bootstrap aggregating	Modelos independientes en submuestras, combinados por promedio o votación.
Bosque aleatorio	Random Forest	Bagging de árboles con selección aleatoria de variables en cada división.
Boosting	Boosting	Modelos secuenciales donde cada uno corrige los errores del anterior.
Gradient Boosting	Gradient Boosting	Boosting que ajusta cada modelo nuevo a los residuos del conjunto acumulado.
XGBoost / LightGBM	—	Implementaciones modernas y optimizadas de gradient boosting.
Caja negra	Black box	Modelo cuyo razonamiento interno es difícil de explicar.
Aprendizaje profundo	Deep learning	Redes neuronales de múltiples capas que aprenden representaciones automáticamente.
Red neuronal artificial	Artificial neural network	Modelo de capas de neuronas interconectadas con pesos ajustables.
Procesamiento del lenguaje natural	Natural Language Processing (NLP)	Disciplina que permite comprender, clasificar y generar lenguaje humano.

Transformador	Transformer	Arquitectura basada en atención que sustenta los modelos de lenguaje actuales.
Visión por computador	Computer vision	Disciplina que interpreta imágenes y video.
Red neuronal convolucional	Convolutional Neural Network (CNN)	Red especializada en detectar patrones espaciales en imágenes.
Aprendizaje por refuerzo	Reinforcement learning	Aprendizaje por interacción con un entorno mediante recompensas y castigos.
Recompensa acumulada	Cumulative reward	Suma de recompensas que el agente busca maximizar a largo plazo.
MSE	Mean Squared Error	Promedio de los errores al cuadrado.
RMSE	Root Mean Squared Error	Raíz del MSE; error promedio expresado en la escala original de los datos.
R-cuadrado	R-squared / coefficient of determination	Proporción de la varianza total explicada por el modelo (entre 0 y 1).
Residuo	Residual	Diferencia entre el valor real y el valor predicho.
Homocedasticidad	Homoscedasticity	Dispersión uniforme (varianza constante) de los residuos: el supuesto deseable.
Heterocedasticidad	Heteroscedasticity	Dispersión no uniforme de los residuos (típico embudo): el problema.
Matriz de confusión	Confusion matrix	Tabla que cruza predicciones y valores reales en TP, TN, FP y FN.
Falso positivo	False positive (Error Tipo I)	Predecir positivo cuando el caso real es negativo (falsa alarma).
Falso negativo	False negative (Error Tipo II)	Predecir negativo cuando el caso real es positivo (omisión).
Exactitud	Accuracy	Proporción total de aciertos; engañosa con clases desbalanceadas.
Precisión	Precision	De los predichos positivos, proporción que era realmente positiva.

Recuperación / sensibilidad	Recall / sensitivity	De los positivos reales, proporción que el modelo logró detectar.
Puntuación F1	F1 score	Media armónica entre precisión y recall; castiga el desbalance entre ambas.
ROC-AUC	ROC-AUC	Área bajo la curva ROC; separación de clases evaluada en todos los umbrales.
Umbral de decisión	Decision threshold	Probabilidad a partir de la cual el clasificador declara un positivo.
Maldición de la dimensionalidad	Curse of dimensionality	Con demasiadas variables los datos se vuelven dispersos y los patrones, indetectables.
Valor atípico	Outlier	Punto extremo que puede sesgar drásticamente el ajuste del modelo.
Meseta de precisión	Accuracy plateau	Punto donde añadir más datos ya no mejora el desempeño pero sí los costos.
Deriva	Drift	Cambio en los datos o en la relación aprendida que degrada el modelo en el tiempo.
MLOps	Machine Learning Operations	Ciclo continuo de despliegue, monitoreo y reentrenamiento de modelos.
Valor de vida del cliente	Lifetime Value (LTV)	Ingreso total que un cliente generará durante su relación con la empresa.
Fuga de clientes	Churn	Abandono del servicio o plataforma por parte del cliente.
Ingeniería de características	Feature engineering	Diseño y construcción de variables predictivas de calidad.

Mapa conceptual

APRENDIZAJE AUTOMÁTICO SUPERVISADO (Unidad 3)

[se monta sobre CRISP-DM — módulo de metodología — y consume los datos cuya calidad definimos con las 5 V y el Ground Truth — módulo de Big Data]

— Fundamento: aprender de ejemplos con respuesta conocida

- Características (features) → señales de entrada
 - Etiqueta (label) = ground truth → lo que se predice
 - numérica continua → REGRESIÓN
 - categórica discreta → CLASIFICACIÓN
- Flujo de trabajo (7 pasos)
 - 1. Recopilar y organizar datos → Tidy Data (fila=observación, columna=variable, celda=valor)
 - 2. Definir features y label → feature engineering > sofisticación del algoritmo
 - 3. Dividir train / (validación) / test → generalizar, no memorizar
 - series temporales: respetar cronología → evitar fuga de datos (leakage)
 - 4. Entrenar y comparar algoritmos → algoritmo ≠ entrenamiento ≠ modelo; hiperparámetros
 - 5. Evaluar → métricas según tipo de problema + costo asimétrico de los errores
 - 6. Implementar (inferencia) → API en tiempo real / batch → output debe volverse outcome
 - 7. Mantener → drift (data drift / concept drift — módulo anterior) → monitoreo → MLOps
- Familias de algoritmos (datos estructurados) — trade-off precisión ↔ interpretabilidad
 - Regresión lineal → ecuación de coeficientes (OLS) → máxima interpretabilidad → línea de base
 - Árboles de decisión → reglas si-entonces (Titanic) → intuitivos, pero sobreajustan e inestables
 - Métodos de conjunto → muchos débiles = uno fuerte → máxima precisión, caja negra
 - Bagging (paralelo, promedio/votación) → Random Forest
 - Boosting (secuencial, corrige errores) → Gradient Boosting, XGBoost, LightGBM
- Datos no estructurados (la Variedad del Big Data) → APRENDIZAJE PROFUNDO
 - Redes neuronales de capas → aprenden las features solas (vs. ingeniería manual)
 - PLN → transformers (atención) → texto corporativo → variables accionables
 - Visión por computador → CNNs (filtros espaciales) → imágenes → información estructurada
- [paradigma alternativo] Aprendizaje por refuerzo → agente + entorno + recompensas
 - sin ground truth predefinido → descubre la mejor secuencia de acciones
- Evaluación (predicciones vs. ground truth en el test set)
 - Regresión → RMSE (escala original) · R^2 (varianza explicada) · residuos
 - nube aleatoria = linealidad + homocedasticidad ⚠ (el PDF la llama mal)
 - Clasificación → matriz de confusión (TP, TN, FP=Tipo I, FN=Tipo II)
 - exactitud → engañosa si las clases están desbalanceadas
 - precisión → prioriza cuando el FP es caro
 - recall → prioriza cuando el FN es caro (fraude, salud, fallas)
 - F1 → media armónica: equilibrio entre ambas
 - ROC-AUC → desempeño en todos los umbrales (0.5 azar, 1.0 perfecto)

- └─ Errores comunes (trampas estratégicas)
 - └─ usar ML sin necesidad → exigir delta sobre una línea de base simple
 - └─ sobreajuste → buscar el sweet spot (errores de train y test bajos y similares)
 - └─ maldición de la dimensionalidad → reducir/combinar variables (precio por m²)
 - └─ ignorar outliers → detectar y decidir (sesgan el OLS)
 - └─ no detenerse con suficientes datos → meseta de precisión, costos de etiquetado
- └─ Casos integradores
 - └─ LTV → regresión + Boosting → RMSE alto → residuos → outliers → segmentar →
R²=0,85
 - └─ NovaMarket (churn) → los 7 pasos completos → baseline 90 días → árbol →
ensembles
 - recall/precisión/F1 sobre accuracy → score semanal → campañas de retención